



PROYECTO SEGUNDO TRIMESTRE

Lenguaje de Marcas

Descripción del Proyecto:

La plataforma **R&M Wiki** te contrata para diseñar y desarrollar una web tipo **wiki** sobre la serie *Rick and Morty*. El proyecto se dividirá en tres fases:

- **Estado inicial:** Diseño de la estructura básica con **HTML y CSS**.
- **Primer incremento:** Incorporación de interactividad mediante **JavaScript**, centrada en **eventos y manipulación del DOM**.
- **Segundo incremento:** Consumo de datos externos usando **fetch()** para obtener y actualizar información desde la **API pública de Rick and Morty**.

Consideraciones IMPORTANTES:

1. El proyecto se entregará vía GitHub: <https://classroom.github.com/a/Chi7p4nX>
2. La gestión de versiones es **obligatoria para que el proyecto sea evaluado**:
 - a. Debemos ver varios commits para comprobar la evolución del proyecto.
 - b. Proyectos realizados por arte de magia, con un único commit final, varios commits con el mismo contenido, pocos commits o commits en un espacio muy breve de tiempo, se entenderán como no realizados por el alumno.
 - c. Si no se cumple este requisito, la calificación del proyecto es de cero (0) puntos.
3. La defensa del proyecto es obligatoria. Si no se realiza la defensa, la calificación del proyecto es de cero (0) puntos.

Estructura de Páginas de la Web

La web debe incluir las siguientes páginas base (se trabajarán por fases según se indique):

- **index.html**: Página principal con artículos destacados de la wiki.
- **detalle.html**: Página de detalle de un artículo (personaje/planeta/episodio).
- **contribuir.html**: Página con formulario para proponer un artículo.
- **sobre_nosotros.html**: Información del proyecto "R&M Wiki".

Navegación entre páginas

- **Menú de navegación (presente en todas las páginas):**
 - **Inicio** → lleva a **index.html**
 - **Sobre nosotros** → lleva a **sobre_nosotros.html**



- Desde **index.html**:
 - Cada artículo destacado tendrá un botón **"Ver más"** → lleva a **detalle.html**
- Desde **detalle.html**:
 - Botón **"Proponer mejora"** → lleva a **contribuir.html**
- Desde **contribuir.html**:
 - El usuario puede volver a la portada mediante el enlace **Inicio** del menú de navegación.
- **Pie de página (en todas las páginas)**:
 - Enlaces a redes sociales
 - Datos de contacto del proyecto.

Requisitos comunes (todas las páginas)

- Mantener **header + menú + footer** iguales en todas las páginas (misma estructura y estilos).
- Aplicar una **paleta de colores coherente** (tema sci-fi / Rick and Morty).
- Usar tamaños de fuente con **rem/em** (nada de todo en **px**).
- Cuidar **márgenes y padding** (espaciado consistente y agradable).
- Botones con:
 - bordes redondeados
 - hover visible (cambio de color/sombra)
- Footer siempre visible y bien separado del contenido (sin "pegotes").

index.html (Portada)

- Convertir el listado en un **layout tipo grid** de tarjetas
- Diseñar la **tarjeta** para que se vea "de catálogo":
 - imagen bien recortada
 - título destacado
 - metadatos (categoría y dato rápido) más discretos
 - botón alineado y con buen contraste
- Duplicar la tarjeta de ejemplo hasta tener **5 tarjetas**.
- Añadir **sombra (box-shadow)** y separación entre tarjetas.



detalle.html (Detalle)

- Maquetar la sección detalle con un diseño claro:
 - imagen + texto bien colocados (columna/filas con flex o grid)
 - texto legible (interlineado, anchura cómoda)
- La lista de datos (categoría/especie/estado) debe verse como “ficha”:
 - con fondo suave o borde
 - buena separación entre items
- Botón “Proponer mejora” bien visible.

contribuir.html (Formulario)

- Formulario con apariencia profesional:
 - labels claros
 - inputs con bordes, padding y foco visible
 - textarea cómoda
- Campo “Nivel de spoiler” maquetado en fila/columna con buen espaciado.
- Botón principal destacado.

sobre_nosotros.html

- Texto con buena lectura:
 - título y subtítulos bien jerarquizados
 - párrafos con separación
- Lista de normas con estilo limpio.
- “Equipo” presentado como tarjetas o bloques simples (mínimo 1, opcional duplicar).



Primer incremento – Interactividad con JavaScript

En esta fase del proyecto, debes añadir interactividad básica a la web R&M Wiki usando JavaScript en un archivo externo llamado [script.js](#)

Requisitos del primer incremento

1. Interactividad en `index.html`

1.1. Botón de favoritos

En cada tarjeta de personaje debes añadir un botón como:

- `Añadir a favoritos`

Cuando pulses el botón:

- el texto debe cambiar, por ejemplo, a `Guardado`
- la tarjeta debe cambiar visualmente para que se note que está marcada como favorita
- si vuelves a pulsarlo, debe poder deshacerse

1.2. Contador de tarjetas

Debes mostrar en la portada un texto generado con JavaScript indicando cuántas tarjetas destacadas hay.

Ejemplo:

- `Personajes destacados: 5`

Ese texto debe insertarse desde `script.js`.



1.3. Ir al detalle del personaje correcto

Cuando pulses en el botón “**Ver más**” de una tarjeta, no debe abrirse siempre el mismo detalle sin más, sino que debes hacer que lleve al **detalle del personaje correspondiente**. Adjunto un anexo donde puedes encontrar como hacerlo.

2. Interactividad en `detalle.html`

2.1. Mostrar u ocultar la ficha

Debes añadir un botón que permita:

- mostrar la ficha del personaje
- ocultarla después si vuelves a pulsarlo

El texto del botón debe ir cambiando según el estado. Ejemplo

- `Mostrar ficha`
- `Ocultar ficha`

2.2. Cargar el personaje correcto según el botón pulsado en la portada

La página `detalle.html` debe mostrar la información del personaje correcto según la tarjeta en la que hayas pulsado en `index.html`.

Debes conseguir que cambien, al menos:

- el nombre del personaje
- la imagen
- algunos datos básicos de la ficha

No hace falta que haya muchísimos personajes distintos, pero sí que se vea claramente que **cada botón de la portada lleva a un detalle diferente**.



Segundo incremento – Consumo de API con `fetch()`

Descripción del incremento

En esta fase del proyecto debes crear una **nueva página** en tu web llamada `buscar_personajes.html`

La página tendrá **dos secciones principales**:

- una sección para **buscar un personaje por nombre o por id**
- otra sección para **mostrar un personaje aleatorio**

1. Nueva página: `buscar_personajes.html`

Os apporto el código html para el incremento

2. Sección de búsqueda por nombre o id

En la parte superior de la página debes incluir un pequeño formulario o zona de búsqueda con:

- un campo de texto
- un botón para buscar

En ese campo el usuario podrá escribir:

- un **nombre**, por ejemplo: `Rick Sanchez`
- o un **id**, por ejemplo: `1`

Cuando pulses el botón de búsqueda, debes hacer una petición a la API y mostrar el resultado en la misma página.



Debes mostrar, como mínimo:

- nombre del personaje
- imagen
- especie
- estado

3. Búsqueda por nombre o por id

Debes detectar si lo que ha escrito el usuario es:

- un número → en ese caso buscarás por **id**
- un texto → en ese caso buscarás por **nombre**

Busca documentacion en la api, son endpoints distintos

4. Mostrar el resultado en pantalla

El personaje encontrado debe aparecer dentro de una zona del HTML preparada para ello. La información no debe mostrarse con `alert()`, sino dentro de una tarjeta, bloque o ficha visual en la propia página. Si la búsqueda es por nombre y la API devuelve varios resultados, puedes mostrar solo el primero.

5. Gestión de errores

Si el personaje no existe o la búsqueda falla, debes mostrar un mensaje visible en la página indicando que no se ha encontrado ningún resultado.

Ejemplos:

- No se ha encontrado ningún personaje con esos datos
- Debes escribir un nombre o un id válido



6. Sección de personaje aleatorio

En otra sección de la misma página debes añadir un botón como:

- **Mostrar personaje aleatorio**

Cuando pulses ese botón:

- debes generar un id aleatorio
- debes hacer una petición a la API con ese id
- debes mostrar la información del personaje obtenido en pantalla

Debes mostrar, como mínimo:

- nombre
- imagen
- especie
- estado

Esta información debe mostrarse en otra zona distinta de la búsqueda normal.